

Test Technique – Développeur Back-End Python (FastAPI)

1. Objectif

Créer une mini API REST de gestion de tickets avec FastAPI. L'objectif est de démontrer la maîtrise de :

- FastAPI
- Modélisation avec Pydantic
- Structuration de code propre
- Tests unitaires
- Documentation API

2. Fonctionnalités attendues

a. Modèle Ticket

Chaque ticket doit contenir :

- id (UUID ou int auto-incrémenté)
- title (str)
- description (str)
- status (enum : "open", "stalled", "closed")
- created_at (datetime)

b. Endpoints à implémenter

Méthode	URL	Description
POST	/tickets/	Créer un ticket
GET	/tickets/	Lister tous les tickets
GET	/tickets/{ticket_id}	Récupérer un ticket
PUT	/tickets/{ticket_id}	Modifier un ticket
PATCH	/tickets/{ticket_id}/close	Fermer un ticket

c. Contraintes techniques

- Python 3.10+
- FastAPI
- Base de données en mémoire avec SQLite via SQLAlchemy
- Validation via Pydantic
- Pas d'authentification
- Pas besoin de persistance des données après redémarrage

d. Tests attendus

Un coverage de tests unitaires de minimum 80%.

3. Documentation attendue

- Utilisation automatique de la doc Swagger (/docs)
- Fichier README.md contenant la description du projet ainsi que les commandes pour installer les dépendances, lancer l'API et lancer les tests

4. Livraison

- Déposer le projet dans un dépôt public GitHub ou GitLab
- Ne pas faire apparaître le nom d'entreprise dans le nom du dépôt ou les fichiers

5. Bonus (facultatif si temps disponible)

- Utilisation de docker ou makefile
- Linting avec ruff ou flake8

6. Critères d'évaluation

Le test sera évalué selon les critères suivants :

a. Qualité du code

- Clarté, lisibilité, nommage pertinent
- Respect des bonnes pratiques Python (PEP8)
- Structuration du projet (même simple)
- Modularité et séparation des responsabilités

b. Fonctionnalité

- L'API répond aux besoins fonctionnels décrits
- Les cas de base fonctionnent sans erreur
- Comportements attendus bien gérés (fermeture de ticket, erreurs, etc.)

c. Documentation

- README clair et suffisant pour exécuter l'API localement
- Swagger (généré par FastAPI) fonctionnel
- Présence d'exemples ou instructions d'utilisation

d. Tests

- Présence d'un coverage de tests $\geq 80\%$
- Facilité d'exécution des tests
- Utilisation de Pytest (ou autre équivalent pertinent)

e. Code review technique

Une revue de code peer-to-peer sera organisée avec le Tech Lead de l'équipe :

- Échange sur les choix d'implémentation
- Compréhension des décisions techniques
- Retour sur l'organisation, les modèles, la gestion des erreurs, etc.

Le but est d'évaluer votre raisonnement technique, pas seulement le code brut.